

电子科技大学

实验报告

学生姓名：吕乐彬

学号：2023080904007

指导教师：李忻洋

一、实验项目名称： 实验3-3：寄生到非代码区段末尾并跳回原入口点；实验3-4：尾区段寄生之扩展区段大小

二、实验目的：

本组实验围绕 PE 文件代码寄生、入口点重定向、节属性修改和尾区段扩展展开。通过实验3-3，掌握将寄生代码写入非代码区段末尾的方法，理解非代码节默认不具备执行权限时会导致访问异常的原因，并能够通过修改节 `Characteristics` 字段为目标节增加可执行权限；同时掌握在寄生代码后添加 `JMP` 指令跳回原入口点的方法，使程序在执行寄生代码后仍能恢复原有执行流程。

通过实验3-4，进一步理解当尾区段原有空洞不足以容纳寄生代码时，为什么需要扩展最后一个节的文件大小；掌握 `VirtualSize`、`SizeOfRawData`、`PointerToRawData`、`FileAlignment`、`SectionAlignment` 和 `AddressOfEntryPoint` 等字段之间的关系；能够完成文件扩展、节大小字段修正、入口点修改、节属性修改和动态调试验证，形成从“结构分析”到“控制流验证”的完整实验闭环。

三、实验原理：

PE 文件运行时，系统会先解析 DOS 头、PE 头、可选映像头和节表，再根据这些结构把文件内容映射到进程虚拟内存中。可选映像头中的 `AddressOfEntryPoint` 决定程序装载后的入口 RVA；节表中的 `RVA`、`VirtualSize`、`PointerToRawData`、`SizeOfRawData` 和 `Characteristics` 则决定节在文件和内存中的位置、大小与访问权限。因此，PE 寄生实验的关键不是单纯写入机器码，而是让“入口点、节大小、节权限和跳转关系”保持一致，使寄生代码既能被加载，又能被执行，并最终跳回原程序流程。

（1）实验3-3：寄生到非代码区段末尾并跳回原入口点

实验3-3的核心是将寄生代码写入非代码节末尾的空洞区域，并修改入口点，使程序启动后先执行寄生代码，再通过 JMP 指令跳回原入口点。非代码节如 `.rdata` 通常只保存只读数据，默认不具备可执行权限。如果只写入机器码并修改入口点，程序可能会因为目标节没有 `IMAGE_SCN_MEM_EXECUTE` 权限而发生访问异常。因此，本实验除了写入 NOP 和回跳 JMP 外，还需要修改目标节的 `Characteristics` 字段，使非代码节具备执行权限。

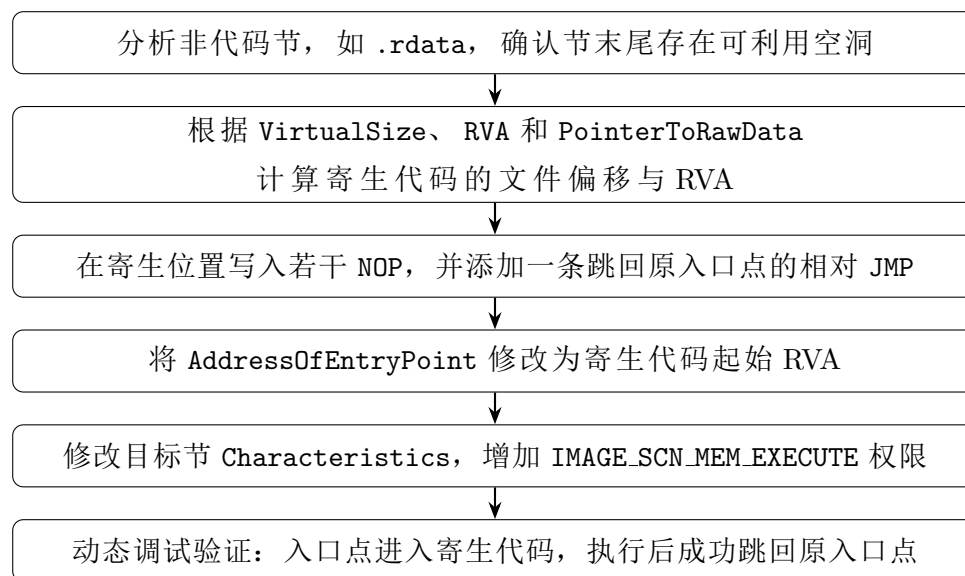


图 1 实验3-3：非代码区段寄生并跳回原入口点的原理流程

如图 1 所示，实验3-3的控制链路为“原入口点被重定向到寄生代码，寄生代码执行后再跳回原入口点”。其中，回跳 JMP 采用相对位移寻址，机器码形式为 `E9 xx xx xx xx`，后 4 字节不是目标地址本身，而是目标 RVA 与 JMP 后下一条指令 RVA 的差值。该实验说明，非代码节寄生必须同时解决两个问题：一是控制流能否跳到寄生位置，二是目标节是否允许执行。

（2）实验3-4：尾区段寄生之扩展区段大小

实验3-4进一步讨论尾区段空洞不足的情况。若寄生代码长度超过最后一个节原有的对齐空洞，仅在原空洞中写入代码已经不够，必须扩展最后一个节的文件大小。具体做法是在最后节原 `VirtualSize` 后写入第一条 JMP，使其跳到原 `SizeOfRawData` 之后的新扩展区域；再在扩展区写入若干 NOP 和一条回跳 JMP。为了使新增内容能够被 PE 加载器识别，还需要同步修改 `SizeOfRawData`、`VirtualSize`，必要时检查 `SizeOfImage`，并补齐新的文件对齐空间。



图 2 实验3-4：尾区段寄生并扩展区段大小的原理流程

如图 2 所示，实验3-4的重点是让新增区域从“文件末尾多出来的字节”变成 PE 结构中合法的节内容。`SizeOfRawData` 决定该节在文件中的对齐后大小，`VirtualSize` 决定该节在内存中的实际有效范围，`SizeOfImage` 则描述整个映像装载后的大小。如果这些字段没有同步修改，即使文件末尾已经写入新代码，加载器也可能不会把它作为有效节内容处理。该实验说明，尾区段寄生超过原空洞时，必须同时处理文件扩展、节大小修正、入口点重定向、节权限修改和跳回原入口点等问题，才能形成完整可靠的执行链路。

四、实验环境（设备、元器件）：

Windows 11 操作系统；PEView；ImHex；x32dbg；实验样本文件 `Control.exe` 及其副本文件。

五、实验步骤：

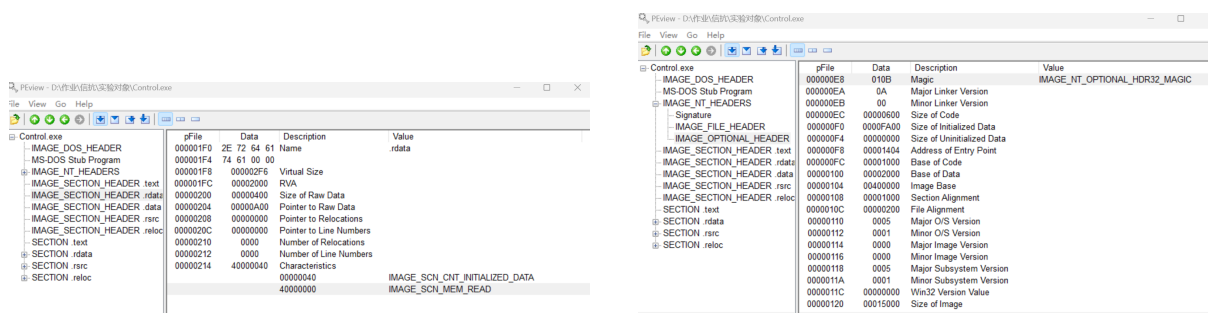
1.1 寄生到非代码区段末尾并跳回原入口点

1.1.1 原始条件分析与寄生思路

本实验要求将寄生代码写入非代码区段末尾，并在执行完寄生代码后跳回原入口点，使原程序继续正常运行。与前一实验不同，本实验的目标不是单纯“修改程序入口点”，而是构造如下控制流：

新入口点 → 寄生代码 → 跳回原入口点 → 原程序继续执行

实验中选取的非代码节为 .rdata。首先使用 PEView 查看原始入口点和 .rdata 节表字段。



(a) PEView 查看 .rdata 节原始字段

(b) PEView 查看原始入口点字段

图 3 原始入口点与 .rdata 节表信息

如图 3 所示，原始样本的 AddressOfEntryPoint 为 0x1404；.rdata 节的 VirtualSize 为 0x02F6，RVA 为 0x2000，Size of Raw Data 为 0x0400，Pointer to Raw Data 为 0x0A00，Characteristics 为 0x40000040。由于 0x02F6 小于 0x0400，说明 .rdata 节末尾存在空洞，可用于写入寄生代码。

空洞大小计算如下：[0x0400 - 0x02F6 = 0x010A] 即 .rdata 节中有 0x010A 字节空闲空间。本实验需要写入 7 个 NOP 和 1 条 5 字节 JMP 指令，共 12 字节，因此空间足够。

1.1.2 寄生位置与跳转位移计算

寄生代码写入位置位于 .rdata 节已使用区域之后，因此其文件偏移为：

$$0x0A00 + 0x02F6 = 0x0CF6$$

对应的寄生代码 RVA 为：

$$0x2000 + 0x02F6 = 0x22F6$$

因此，本实验应将入口点字段改为 0x22F6。由于 PE 文件使用小端序存储多字节整数，0x000022F6 在文件中的字节形式为：

F6 22 00 00

根据学号最后一位数字为 7，寄生代码开头写入 7 个 NOP 指令：

90 90 90 90 90 90 90

随后添加一条相对 JMP 指令，使控制流跳回原入口点 0x1404。JMP 指令开始 RVA 为：

$$0x22F6 + 7 = 0x22FD$$

JMP 指令长度为 5 字节，因此 JMP 后下一条指令 RVA 为：

$$0x22FD + 5 = 0x2302$$

相对位移为：

$$0x1404 - 0x2302 = -0x0EFE$$

写成 32 位补码为：

$$0xFFFFF102$$

按小端序写入后，位移字节为：

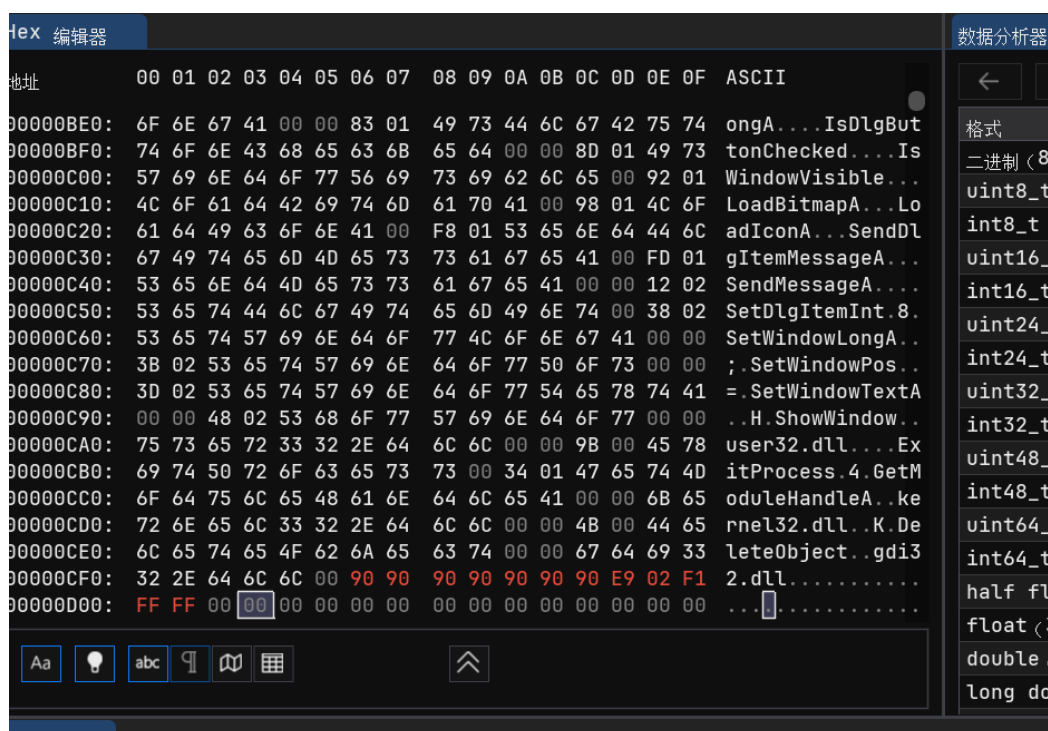
$$02\ F1\ FF\ FF$$

因此完整寄生代码为：

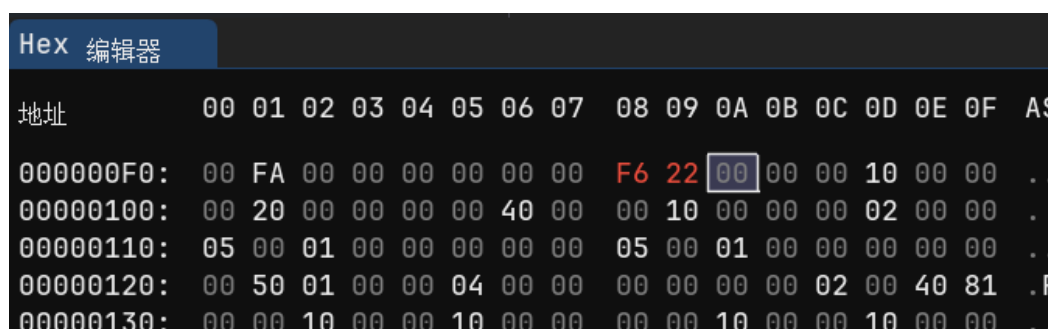
$$90\ 90\ 90\ 90\ 90\ 90\ 90\ 90\ E9\ 02\ F1\ FF\ FF$$

1.1.3 寄生代码写入与关键字段修改

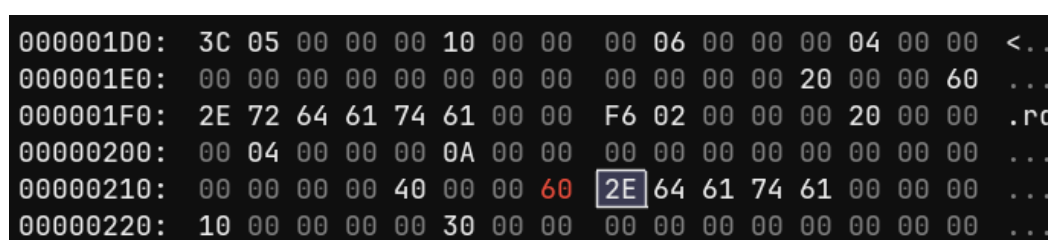
完成地址与位移计算后，使用 ImHex 对样本文件进行了三处关键修改。第一，在文件偏移 0x0CF6 处写入寄生代码；第二，将 AddressOfEntryPoint 从 0x1404 修改为 0x22F6；第三，将 .rdata 节 Characteristics 从 0x40000040 修改为 0x60000040，使该节具有可执行权限。



(a) 在 .rdata 节末尾写入 7 个 NOP 与跳回原入口点的 JMP 指令



(b) 将 AddressOfEntryPoint 修改为寄生代码 RVA 0x22F6



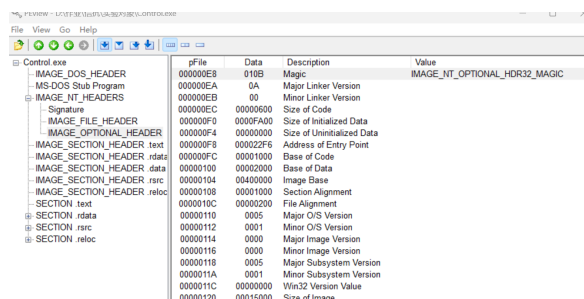
(c) 将 .rdata 节属性由 0x40000040 修改为 0x60000040

图 4 寄生到非代码区段末尾实验中的三处关键修改

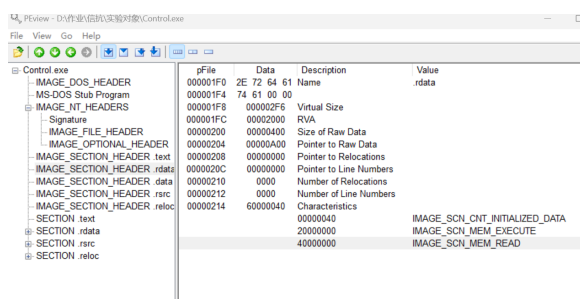
如图 4 所示，寄生代码写入位置为文件偏移 0x0CF6，写入内容为 7 个 NOP 和一条跳回原入口点的 JMP 指令；入口点字段被修改为 F6 22 00 00，对应 RVA 0x22F6；.rdata 节属性被修改为 0x60000040，表示该节增加了可执行权限。三处修改分别解决了“代码放在哪里”“程序从哪里开始执行”和“非代码节能否执行”的问题。

1.1.4 静态结构验证

完成修改后，使用 PEView 对关键字段进行静态验证，确认入口点字段与 .rdata 节属性均已被正确修改。



(a) PEView 验证入口点字段已修改为 0x22F6



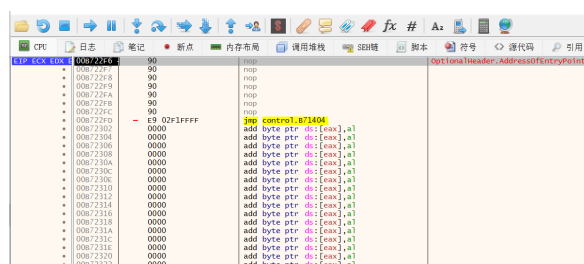
(b) PEView 验证 .rdata 节属性已修改为 0x60000040

图 5 入口点与 .rdata 节属性的静态验证结果

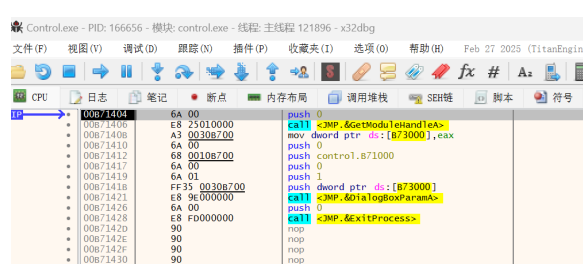
如图 5 所示，左图中 AddressOfEntryPoint 已由原始的 0x1404 修改为 0x22F6，说明程序入口点已经被重定向至 .rdata 节末尾的寄生代码位置；右图中 Characteristics 已变为 0x60000040，其中包含可执行与可读属性，说明该非代码节已经具备执行条件。

1.1.5 动态调试验证

随后使用 x32dbg 对程序执行过程进行动态验证，观察程序是否会首先进入 .rdata 节中的寄生代码，并在执行完寄生代码后跳回原始入口点继续运行。



(a) x32dbg 验证程序首先进入寄生代码位置



(b) x32dbg 验证寄生代码执行后跳回原入口点

图 6 寄生代码执行与跳回原入口点的动态验证结果

如图 6 所示，左图中程序加载后首先停在地址 00B722F6 处，当前位置对应新的入口点 RVA 0x22F6。该位置处可见前 7 条指令均为 nop，之后是一条 jmp control.B71404 指令。该结果表明程序已经按预期从 .rdata 节末尾的寄生代码开始执行。

右图中，当单步执行完 jmp 指令后，控制流成功跳转至 00B71404，该位置对应原始入口点 RVA 0x1404。此时可以看到程序继续执行原始入口处的指令序列，说明寄生代码并未中断程序主流程，而是实现了“先执行寄生代码，再返回原入口点”的设计目标。

综合静态与动态验证结果可以确认：本实验成功将寄生代码写入非代码区段 `.rdata` 的末尾，通过修改入口点字段和节属性，使程序启动后先执行 7 个 `nop`，再借助 `jmp` 指令跳回原入口点继续运行，从而完整实现了“寄生到非代码区段末尾并跳回原入口点”的实验要求。

1.2 尾区段寄生之扩展区段大小

1.2.1 实验目标与扩展寄生思路

前一实验中的尾区段寄生利用的是最后一个节内部已有的对齐空洞，文件长度不发生变化。但在实际情况下，如果寄生代码长度超过最后一个节原有空洞，就不能只在原空洞中覆盖写入，而需要扩展最后一个节在文件中的大小。本实验模拟“寄生代码超过尾区段原有剩余空间”的情况，通过扩展 `.reloc` 节的 `SizeOfRawData`，在原文件末尾之后增加一个文件对齐粒度的空间，并将寄生代码写入新增区域。

本实验构造的控制流如下：

入口点 -> `.reloc` 原 `VirtualSize` 后的 `JMP` -> 新增扩展区中的 `NOP` -> `JMP` 跳回
原入口点

该过程不仅需要写入机器码，还必须同步修改 PE 结构字段，使新增内容能够被 PE 加载器识别和装载。涉及的关键字段包括 `VirtualSize`、`SizeOfRawData`、`AddressOfEntryPoint` 和节属性 `Characteristics`。

1.2.2 原始字段与扩展空间计算

根据样本文件的 PEView 分析结果，原始 `.reloc` 节的关键字段如下：

```
VirtualSize = 0x0122,  RVA = 0x14000,  SizeOfRawData = 0x0200,
PointerToRawData = 0x10000
```

本实验将第一条跳转指令写在 `.reloc` 节原始 `VirtualSize` 之后，其文件偏移为：

$$0x10000 + 0x0122 = 0x10122$$

对应的 RVA 为：

$$0x14000 + 0x0122 = 0x14122$$

因此，本实验需要将程序入口点修改为 `0x14122`，使程序启动后首先执行该位置的第一条 `JMP` 指令。

第一条 `JMP` 的目标是新增扩展区域的起始位置。原 `SizeOfRawData` 为 `0x0200`，因此扩展区起始文件偏移为：

$$0x10000 + 0x0200 = 0x10200$$

对应的 RVA 为:

$$0x14000 + 0x0200 = 0x14200$$

第一条 JMP 从 0x14122 跳转到 0x14200。由于 JMP 指令长度为 5 字节，因此相对位移为:

$$0x14200 - (0x14122 + 5) = 0x00D9$$

所以第一条 JMP 的机器码为:

E9 D9 00 00 00

在扩展区 0x14200 处写入 7 个 NOP，随后写入一条 JMP 指令跳回原入口点 0x1404。
第二条 JMP 的开始 RVA 为:

$$0x14200 + 7 = 0x14207$$

其后一条指令 RVA 为:

$$0x14207 + 5 = 0x1420C$$

因此跳回原入口点的相对位移为:

$$0x1404 - 0x1420C = -0x12E08$$

写成 32 位补码为:

0xFFFFD1F8

按小端序写入后，第二条 JMP 的位移字节为:

F8 D1 FE FF

所以扩展区写入的完整机器码为:

90 90 90 90 90 90 90 E9 F8 D1 FE FF

1.2.3 文件扩展与关键字段修改

完成计算后，使用 ImHex 对样本文件进行修改。首先在文件偏移 0x10122 处写入第一条 JMP 指令 E9 D9 00 00 00，使入口点处的执行流跳转到新增扩展区域。随后将文件扩展到 0x10400，也就是在原文件末尾之后增加一个 FileAlignment 大小的空间。扩展完成后，在文件偏移 0x10200 处写入 7 个 NOP 和一条跳回原入口点的 JMP 指令。

由于新增内容已经超过原 SizeOfRawData 范围，需要将 .reloc 节的 SizeOfRawData 从 0x0200 修改为 0x0400。同时，新增代码的有效末尾到达：

$$0x0200 + 7 + 5 = 0x020C$$

因此将 VirtualSize 修改为 0x020C。最后，将可选映像头中的 AddressOfEntryPoint 修改为第一条 JMP 的 RVA，即 0x14122。

```

00010110: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010120: 00 00 E9 D9 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010130: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

(a) 在 0x10122 处写入第一条 JMP 指令

```

Hex 编辑器
地址 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F ASCII
00010180: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010190: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000101A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000101B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000101C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000101D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000101E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000101F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010200: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010210: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010220: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010230: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010240: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010250: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010260: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010270: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010280: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010290: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000102A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

(b) 扩展文件并补齐新的文件对齐区域

```

000101F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010200: 90 90 90 90 90 90 90 E9 F8 D1 FE FF 00 00 00 00 .....
00010210: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010220: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010230: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010240: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00010250: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

(c) 在 0x10200 处写入 7 个 NOP 与回跳 JMP

```

Hex 编辑器
地址 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F ASCII
00000270: 0C 02 00 00 00 40 01 00 00 04 00 00 00 00 01 00 ..@.....
00000280: 00 00 00 00 00 00 00 00 00 00 00 40 00 00 42 .....@.B
00000290: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000002A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000002B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

(d) 修改 VirtualSize 和 SizeOfRawData

```

地址 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F ASCII
000000F0: 00 FA 00 00 00 00 00 00 22 41 01 00 00 10 00 00 .....A.
00000100: 00 20 00 00 00 00 40 00 00 10 00 00 00 02 00 00 .....@.....
00000110: 05 00 01 00 00 00 00 00 05 00 01 00 00 00 00 00 .....
00000120: 00 50 01 00 00 00 04 00 00 00 00 00 02 00 40 81 P

```

(e) 修改入口点字段为 0x14122

图 7 尾区段扩展寄生实验中的文件扩展与关键字段修改

如图 7 所示，本实验依次完成了第一条跳转指令写入、文件对齐区域扩展、新增区域代码写入、节大小字段修改和入口点字段修改。其中，VirtualSize 被修改为 0x020C，SizeOfRawData 被修改为 0x0400，AddressOfEntryPoint 被修改为 0x14122。这些修改共同保证新增扩展区域能够被 PE 加载器识别，并使程序启动后首先进入寄生代码区域。

1.2.4 首次动态验证与异常原因分析

完成上述修改后，使用 x32dbg 打开修改后的程序。程序加载后首先停在新的入口点位置，对应 RVA 0x14122。该位置处可以看到第一条 JMP 指令，其目标为扩展区域起始地址。然而，直接执行时程序进入异常处理流程，说明当前修改仍不完整。

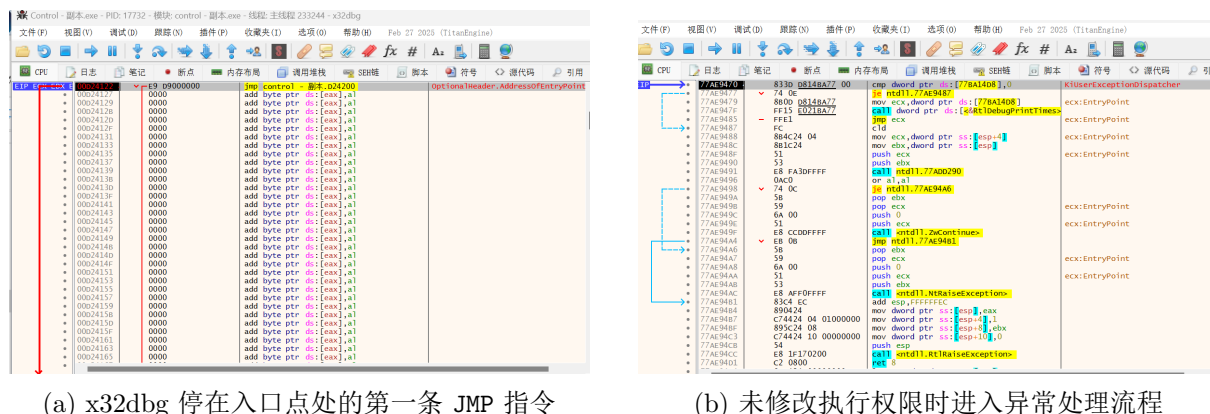


图 8 未修改 .reloc 节执行权限时的异常现象

如图 8 所示，左图中程序已经停在 OptionalHeader.AddressOfEntryPoint 指向的位置，说明入口点修改是有效的；但右图显示程序执行后进入 KiUserExceptionDispatcher，说明在执行 .reloc 节中的代码时发生了访问异常。

产生该问题的原因在于，.reloc 节原本是重定位相关的数据节，并不是代码节。其原始属性为 0x42000040，不包含可执行权限。虽然本实验已经将代码写入 .reloc 节并修改了入口点，但如果该节在内存中没有 MEM_EXECUTE 权限，CPU 仍无法正常运行其中的指令。因此，需要继续修改 .reloc 节的 Characteristics 字段，为其增加可执行权限。

1.2.5 修改 .reloc 节执行权限

为解决访问异常问题，将 .reloc 节的 Characteristics 从原来的 0x42000040 修改为 0x60000040。该修改保留了初始化数据和可读属性，同时增加了内存可执行属性，使 .reloc 节中的寄生代码能够被正常执行。

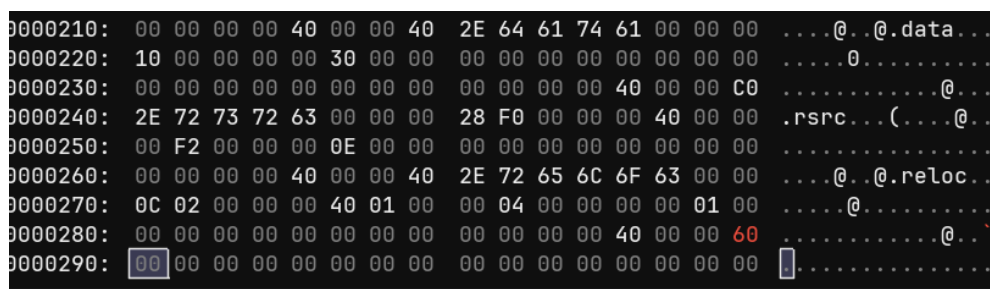


图 9 使用 ImHex 修改 .reloc 节属性为 0x60000040

如图9所示,在 .reloc 节的 Characteristics 字段处,将其字节改为对应 0x60000040 的小端序形式。该步骤的作用是为寄生代码所在节增加执行权限,避免再次进入访问异常。

随后使用 PEView 重新查看 .reloc 节属性,确认修改是否成功。

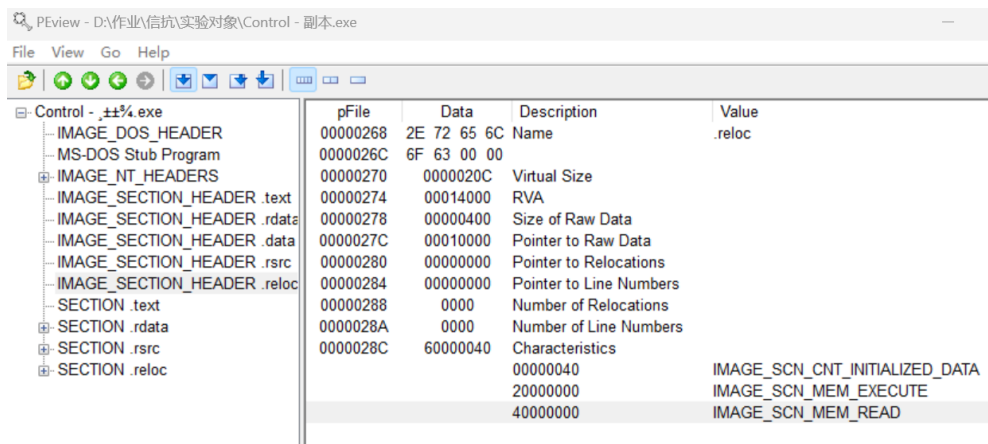


图 10 PEView 验证 .reloc 节已具备可执行权限

如图 10 所示, .reloc 节的 Characteristics 已经变为 0x60000040, 并显示出 IMAGE_SCN_MEM_EXECUTE 和 IMAGE_SCN_MEM_READ 等属性。这说明 .reloc 节已经具备执行寄生代码的权限。

1.2.6 PE 结构最终验证

在完成文件扩展、节大小字段修改、入口点修改和节属性修改后,重新使用 PEView 对 PE 结构进行最终验证。

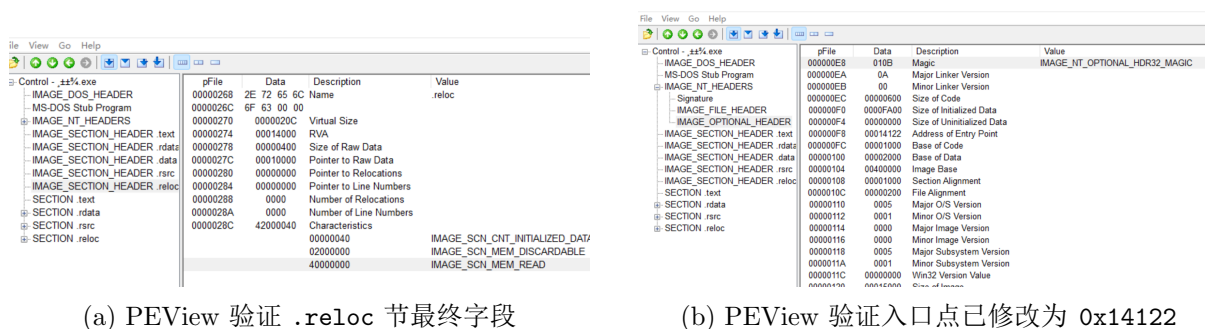


图 11 扩展尾区段后的 PE 结构最终验证

如图 11 所示,左图中 .reloc 节的 VirtualSize 已变为 0x020C, SizeOfRawData 已变为 0x0400, 并且 Characteristics 已变为 0x60000040; 右图中 AddressOfEntryPoint 已变为 0x14122。这些结果说明扩展尾区段寄生所需的 PE 结构字段已经全部修改完成。

另外,根据原始 SizeOfImage 为 0x15000, 扩展后的节尾为:

$$0x14000 + 0x020C = 0x1420C$$

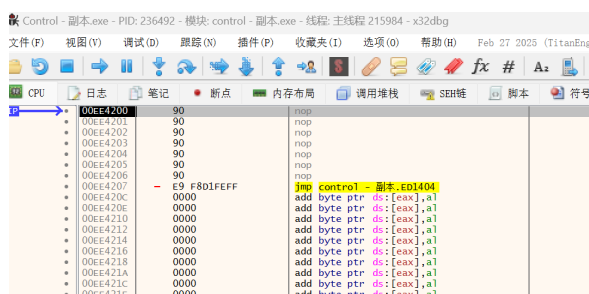
按照 `SectionAlignment = 0x1000` 向上对齐后仍为：

0x15000

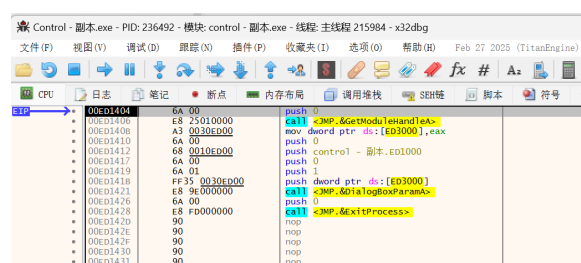
因此本实验中 `SizeOfImage` 不需要修改。

1.2.7 动态调试验证

最后使用 x32dbg 对扩展寄生后的控制流进行动态验证。程序入口首先指向 0x14122，执行第一条 JMP 后跳转到扩展区 0x14200，随后执行 7 个 NOP，再通过第二条 JMP 跳回原入口点 0x1404。



(a) x32dbg 验证程序跳转到扩展区并执行 7 个 NOP



(b) x32dbg 验证第二条 JMP 成功跳回原入口点

图 12 扩展尾区段寄生代码的动态执行与跳回原入口点验证

如图 12 所示，左图中程序已经跳转到扩展区域起始地址，对应 RVA 0x14200，该位置显示 7 条 nop 指令，随后是一条跳回原入口点的 jmp 指令。右图中执行该 jmp 后，程序成功跳回原入口点位置，对应 RVA 0x1404，该处机器码为 6A 00，反汇编结果为 push 0。这说明扩展尾区段中的寄生代码已经被成功加载并执行，且最终能够恢复原程序入口流程。

六、实验结论：

本组实验完成了实验3-3和实验3-4两个 PE 文件寄生任务，分别验证了非代码区段寄生与尾区段扩展寄生两种典型情况。

在实验3-3中，成功将寄生代码写入 .rdata 节末尾空洞。根据 .rdata 节的 `PointerToRawData = 0x0A00` 和 `VirtualSize = 0x02F6`，计算得到寄生代码文件偏移为 0x0CF6，对应 RVA 为 0x22F6。实验中将 `AddressOfEntryPoint` 从原始 0x1404 修改为 0x22F6，并在寄生位置写入 7 个 NOP 和一条跳回原入口点的 JMP 指令。其完整机器码为 90 90 90 90 90 90 90 E9 02 F1 FF FF。同时，将 .rdata 节的 `Characteristics` 从 0x40000040 修改为 0x60000040，使原本不可执行的非代码节具备执行权限。PEView 静态验证显示入口点和节属性均已修改成功，x32dbg 动态验证显示程序能够先进入 .rdata 节末尾的寄生代码，执行后再跳回原入口点 0x1404，说明实验3-3达到预期目标。

在实验3-4中，成功完成了尾区段寄生时扩展区段大小的过程。实验以 `.reloc` 节作为尾区段，原始 `VirtualSize` 为 `0x0122`，`RVA` 为 `0x14000`，`SizeOfRawData` 为 `0x0200`，`PointerToRawData` 为 `0x10000`。第一条 `JMP` 被写入文件偏移 `0x10122`，对应 `RVA` 为 `0x14122`，并将程序入口点修改为该 `RVA`。随后将文件扩展到 `0x10400`，在扩展区文件偏移 `0x10200` 处写入 7 个 `NOP` 和一条跳回原入口点的 `JMP` 指令，完整机器码为 `90 90 90 90 90 90 E9 F8 D1 FE FF`。为保证新增内容属于合法节范围，实验将 `.reloc` 节的 `SizeOfRawData` 从 `0x0200` 修改为 `0x0400`，将 `VirtualSize` 修改为 `0x020C`。根据计算，扩展后的节尾为 $0x14000 + 0x020C = 0x1420C$ ，按 `SectionAlignment = 0x1000` 对齐后仍未超过原 `SizeOfImage = 0x15000`，因此 `SizeOfImage` 不需要修改。

实验3-4首次动态调试时出现异常，原因是 `.reloc` 节原属性 `0x42000040` 不包含可执行权限。随后将 `.reloc` 节的 `Characteristics` 修改为 `0x60000040`，使其具备 `IMAGE_SCN_MEM_EXECUTE` 和 `IMAGE_SCN_MEM_READ` 属性。最终，`x32dbg` 验证程序能够在新入口点 `0x14122` 执行第一条 `JMP`，跳转到扩展区 `0x14200`，执行 7 个 `NOP` 后，再通过第二条 `JMP` 成功返回原入口点 `0x1404`。由此说明，实验3-4完整实现了“入口点重定向、尾区段扩展、扩展区代码执行、跳回原入口点”的控制流闭环。

七、总结及心得体会：

通过本组实验，我对 PE 文件中“结构字段”和“实际执行流程”之间的关系有了更清晰的理解。PE 文件不是简单的字节序列，程序运行时系统会依据 PE 头和节表进行装载。因此，修改 PE 文件时不能只关注某一处机器码是否写入成功，还必须检查节表字段、入口点字段和节属性是否与写入内容保持一致。实验3-3和实验3-4正好从两个不同角度说明了这一点：实验3-3突出的是非代码节的执行权限问题，实验3-4突出的是新增内容超出原节范围后的结构修正问题。

在实验3-3中，最关键的认识是：非代码节中即使存在空洞，也不意味着写入其中的字节就可以直接执行。`.rdata` 节原本用于保存只读数据，不具备执行权限。如果只修改 `AddressOfEntryPoint`，程序虽然能够从逻辑上跳到寄生代码位置，但在真正执行时仍可能因为内存页权限不足而触发异常。因此，`Characteristics` 字段不是一个附属字段，而是直接影响程序运行结果的关键字段。通过将 `.rdata` 节属性修改为 `0x60000040`，我理解了 PE 文件中节权限对代码执行的重要作用。

在实验3-4中，最大的体会是尾区段寄生超过原空洞后，问题会从“找地方写代码”变成“让新增内容成为合法节内容”。如果只在文件末尾添加字节，但不修改 `SizeOfRawData`，新增内容在文件结构上不属于该节；如果不修改 `VirtualSize`，新增内容在内存映像中也不能被视为该节的有效范围。实验中还需要根据 `SectionAlignment` 检查 `SizeOfImage` 是否变化，这说明 PE 文件修改必须同时考虑文件视角和内存视角。通过这一过程，我进一步理解了 `FileAlignment` 和 `SectionAlignment` 的区别：前者

约束文件中的节大小对齐，后者约束内存中的节映射边界。

这两个实验还让我认识到，相对 JMP 指令的位移计算是控制流恢复的关键。无论是在 .rdata 节末尾跳回原入口点，还是在扩展区执行后跳回原入口点，E9 后面的 4 字节都不是目标地址，而是相对位移。若没有考虑 JMP 指令本身的 5 字节长度，就会导致跳转目标错误，程序无法回到原流程。因此，实验中的每一步计算都必须与 x32dbg 中的实际地址相互验证，不能只凭直觉修改字节。

从安全分析角度看，这组实验也提供了识别 PE 寄生行为的思路。若一个 PE 文件的入口点异常指向非代码节或尾区段，某个数据节突然被赋予 IMAGE_SCN_MEM_EXECUTE 权限，或者最后一个节的 SizeOfRawData、VirtualSize 出现异常扩展，就需要警惕其是否存在寄生代码或控制流劫持行为。也就是说，实验中的修改过程反过来就是恶意样本分析中的检查线索。

报告评分：

指导教师签字：